

École polytechnique de Louvain

Profiling Novice Programmers' Behavior through the Analysis of Incremental Programming Submissions

Author: **François Junior MELI NGUEUNKEUNG**

Supervisors: **Kim MENS, Siegfried NIJSSEN**

Co-supervisor: **Guillaume STEVENY**

Reader: **Olivier GOLETTI**

Academic year 2025–2026

Master [120] in Computer Science and Engineering

Abstract

Novice programmers face significant challenges when learning to write code. Understanding how students develop solutions incrementally through iterative submissions can help identify misconceptions, ineffective learning strategies, and opportunities for intervention. This thesis provides a framework for computer science educators to support student learning through data-driven insights into learner behavior and problem-solving processes. We do so by analyzing incremental programming submissions from an introductory Python course at UCLouvain. Using submission history data from the INGIInious autograder, we apply a mixed-method approach combining quantitative clustering, qualitative case studies, and student self-report surveys. Based on submission patterns, iteration rhythm, and temporal pacing, our study identifies two main behavioral characteristics of different learner groups. It later appears that the quality of the iterations and the consistency of the work rhythm are strong predictors of success, as shown by the quantitative results. A close look at the code, on the other hand, shows how by focusing on the concepts understanding, it can help with problem-solving patterns.

Keywords: novice programming, behavioral profiling, educational data mining, hierarchical clustering, mixed-methods analysis, introductory computing education.

Acknowledgment

I would like to thank the supervisors of this thesis: Prof. Kim Mens and Prof. Siegfried Nijssen for their consistency and my guidance throughout this thesis. Across weekly meetings, continuous support and feedbacks, Prof. Mens provided invaluable advice on research design and methodology, while Prof. Nijssen contributed via its expertise in AI and machine learning.

I also want to thank Guillaume Steveny who offered valuable perspective and overall advice alongside Prof. Kim Mens and Prof. Siegfried Nijssen.

I am also grateful to Dr. Olivier Goletti for his advice during the Poster session presentation, which proved helpful for the qualitative analysis of this master thesis. Special thanks to Julien Liénard for providing the first versions of the INGINIOUS data extraction, which laid the foundation for this thesis.

I thank the students who completed the survey and generously dedicated their time and insights. Their responses helped me update and enhance the overall analysis in this thesis.

Furthermore, I am deeply grateful to my family, partner, and friends for their encouragement, support, and confidence throughout this work.

Finally, this thesis reflects the belief that understanding how novice programmers think and learn can benefit computer science education.

Disclaimer

This thesis relies on the Python scientific stack for data processing and visualization and on historical submission data from the INGIInious autograder platform.

GitHub Copilot assisted with code generation for data processing utilities, profile denomination suggestions, and tone refinement across chapters to ensure consistency and clarity.

All analytical choices, cluster interpretation, statistical decisions, pedagogical recommendations, and final research conclusions remain my own. The analysis and findings reflect my independent understanding and assessment of the data and literature.

Contents

1	Introduction	7
1.1	Research Context	7
1.2	Research Questions	8
1.2.1	RQ1: What behavioral profiles exist?	8
1.2.2	RQ2: Why do students adopt these behaviors?	8
1.2.3	RQ3: How do profiles and strategies align?	8
1.3	Contributions	9
1.4	Obtained Results	9
1.5	Thesis Structure	10
2	Background and Related Work	11
2.1	Novice Programmer Behavior	11
2.2	Educational Data Mining in Programming Education	12
2.3	Temporal Learning Analytics	12
2.4	Behavioral Profiling and Clustering in CS Education	13
2.4.1	Qualitative Observations of Coding in the Wild	13
2.5	Research Gap	14
3	Context and Data	15
3.1	Course and Platform Context	15
3.2	Data Overview	16
3.3	How the Code Processes the Data	16
4	Quantitative Results	17
4.1	Methodology: How We Analyzed Student Behavior	17
4.2	Finding our optimal number of clusters and profiles	18
4.2.1	Elbow method and silhouette score	22
4.2.2	Proportion of profiles accross $k = 2$ clusters	24
4.3	Further observations	25

4.3.1	Demystifying a myth: "Question difficulty increases the higher the question index is"	25
4.3.2	Submission Trajectories	26
4.4	Grades, Speed, and Persistence Across the Clusters	28
4.4.1	Final Scores and Success Rates	28
4.4.2	How Students Pace Their Attempts	28
4.5	How Questions Link Together	28
4.6	Students Stuck in the Middle	29
4.7	Time Analysis: How Students Split Their Time and Energy	30
4.7.1	Time Spent on Each Question	30
4.7.2	Does Spending More Time Mean Better Grades?	31
4.7.3	Exam Progression and the Impact of Fatigue or External Factors	33
4.8	Quantitative Summary	34
5	Qualitative Results	36
5.1	How the cases were chosen	36
5.1.1	What we look for in the code	37
5.2	Representative Student profiles	38
5.2.1	Elite Learner	38
5.2.2	Quick Success	38
5.2.3	Persistent Learner	39
5.2.4	Checked Out	39
5.2.5	Struggling Guesser	40
5.2.6	Thrashing	41
5.3	Answering RQ2: Why Students Adopt These Behaviors	41
5.3.1	Extreme reliance on the Autograder as an Interpreter	41
5.3.2	Getting Stuck on the Same Logic	42
5.3.3	Giving Up Early to Save Time	42
5.3.4	Having a Clear Plan From the Start	42
6	Survey Results and Teaching Tips	43
6.1	Survey Results: Where and why Students Get Stuck	43
6.1.1	Survey Design	43
6.1.2	The Main Weak Spot: Understanding Loops	45
6.1.3	First-Year Students Struggle the Most	45
6.2	Matching Student Profiles to Teaching Advice	48

6.3 Chapter Summary	49
7 Conclusion	50
7.1 Answer to RQ1: What behavioral profiles exist?	50
7.2 Answer to RQ2: Why do students adopt these behaviors?	51
7.3 Answer to RQ3: How do profiles and strategies align?	51
7.4 Limits of this study	51
7.5 Future work	52
References	53
A Reproducibility Appendix	55
B Additional Visualizations	56
C Statistical Tables	57
D Glossary	58

Chapter 1

Introduction

Learning is a continuous process that relies on feedback. Students use feedback to assess their progress, identify strengths, and recognize weaknesses. Without such constructive comments, students cannot realize where their approach needs adjustment nor understand why.

I believe that nothing is more normal than making mistakes during learning, and errors become learning opportunities when they are properly contextualized and explained.

At university, many courses require learning to program. Students practice autonomously through exercises and projects, developing skills incrementally. However, providing personalized feedback becomes increasingly difficult for instructors as class sizes grow and course complexity increases. This challenge is not merely logistical, but it stems from a fundamental resource constraint: teachers lack sufficient means to give meaningful, individual follow-up to every student. Understanding how students develop solutions through iterative attempts can help address this gap.

1.1 Research Context

Learning to program is not a single event but a sequence of decisions, corrections, hesitations, and reformulations. In introductory courses, students often submit many intermediate versions before reaching a final answer, and these iterations carry valuable information about how they reason, how they debug, and how they persist when they face difficulty. Yet, in many practical settings, this temporal richness is collapsed into final marks or *binary success indicators*¹. As a consequence, teachers and tutors can observe that some students struggle, but they do not always have

¹By *binary success indicator* we mean an evaluation that records only final correctness (e.g. failed/succeeded, green display/red display) and ignores intermediate attempts.

operational tools to characterize *how* or *why* they struggle, and therefore *how* and *when* to intervene.

This thesis is motivated by that gap between available traces and usable insights for teaching. The INGINious² autograder platform stores detailed submission histories, and these histories offer an opportunity to move from static evaluation toward behavioral interpretation. The central intuition of this work is that student trajectories can be profiled through a combination of outcome, effort, and timing signals, and that such profiles can help bridge analytics and instructional action. The objective is therefore not only predictive. The objective is interpretive and actionable: to design an analysis workflow that remains scientifically grounded while producing patterns that educators can read, question, and eventually use in practice.

1.2 Research Questions

The study is structured around three complementary research questions. Together, they articulate the quantitative, qualitative, and integrative dimensions of the thesis.

1.2.1 RQ1: What behavioral profiles exist?

Can we identify interpretable groups of students from their submission histories using quantitative methods that combine performance and timing? The analysis uses feature engineering, clustering, and standard validation (e.g., silhouette/elbow, cluster summaries) to describe the resulting profiles.

1.2.2 RQ2: Why do students adopt these behaviors?

What variables or behaviors explain why students take successful or unsuccessful paths? This analysis uses case studies, survey responses, and anonymized code snippets of students' submission histories to connect quantitative behavioral profiles with the most plausible explanations.

1.2.3 RQ3: How do profiles and strategies align?

To what extent do quantitative partitions and qualitative interpretations reinforce each other, and where do they diverge? This final question serves as a methodological consistency check: if the two perspectives systematically agree, this can serve

²INGInious autograder documentation: https://docs.inginius.org/en/latest/what_is_inginius.html.

as supporting proof of their conclusions, if not, the the profile conclusions would have to be revisited.

1.3 Contributions

This thesis makes three practical contributions:

First, we built a step-by-step process that turns student submissions into clear descriptions of how they work and how they behave. The process starts with raw data (submissions files from INGINious), extracts meaningful measurements (how many attempts, how long they wait between tries, what scores they get), groups students who behave similarly, and produces visualizations that show the patterns. Second, we created a simple vocabulary for talking about student behavior in teaching terms. Rather than just saying "Cluster A" or "Cluster B", we use four dimensions: Do they persist (keep trying)³? Do they succeed (high scores)⁴? Do they iterate (try multiple approaches)⁵? Are they engaged (regular rhythm)⁶? These four yes/no questions create 16 possible profiles of students. This vocabulary is meant to help teachers understand the data in a way that's actionable for teaching.

Finally, we conducted qualitative analyses by examining the actual code content submitted by students. This required identifying interesting patterns and practices that correlate with our quantitative results. We believe that by studying the coding approaches and mistakes made by students, we might get useful insights that improves the interpretation of the quantitative data.

1.4 Obtained Results

We produced in this thesis a reproducible analysis pipeline along with a study of typical student behaviors based on their submissions. The pipeline transforms raw submission data into profiles and visual summaries, while the qualitative analysis helps explain what those behaviors mean in practice.

All source code, preprocessing scripts, an useful utilities are documented and version-controlled. The complete repository is available on Github⁷.

³Which will later be associated with the `*attempts` features

⁴Which will later be associated with the `*best_score` features

⁵Which will later be associated with the `*fast_retry_ratio` features

⁶Which will later be associated with the `*long_pause_ratio` features

⁷https://github.com/Lord-Melflam/Master_Thesis_Francois_Meli_2026.

1.5 Thesis Structure

The manuscript is organized around chapters:

Chapter 2 explains the background. It describes what we know about how students learn to program, what other researchers have discovered about this problem, what gaps still exist in our understanding and why this research matters.

Chapter 3 describes the context and data we are studying. We show the course where the data comes from, explain how the *INGInious* platform works (the autograder that students submit code to), and describe exactly what submission data we collected and how it is organized.

Chapter 4 presents our quantitative methodology and findings. The patterns we discovered when we analyzed thousands of submissions. This chapter starts by explaining how we turned raw submissions into measurements we could analyze, then it shows the student profiles we discovered and uses visualizations to demonstrate the patterns we’ve found.

Chapter 5 adds a qualitative perspective by examining actual code from individual students. We start by showing our approach: how we chose which students to study in detail and why we chose to read their code. Then we show concrete code examples from different student profiles, explaining what each example teaches us about how and why students adapt specific behaviors.

Chapter 6 looks at the results of a real student survey we did to support our thesis by connecting our quantitative and qualitative findings. We then discuss what these results mean for teaching, showing which specific interventions can help instructors support students, based on their profiles.

Chapter 7 concludes the thesis by directly answering our research questions, explaining the limitations of our study, and suggesting directions for future work.

Chapter 2

Background and Related Work

This chapter reviews the literature and concepts needed as theoretical background for this thesis. The first section discusses novice programmer behavior and why tracking incremental code submissions reveals learning patterns that final scores hide. Next, we cover educational data mining, focusing on the balance between model accuracy and how easy the results are for teachers to interpret.

After that, we introduce temporal learning analytics to show how the timing and rhythm of student entries serve as behavioral signals. We then examine clustering techniques in computer science education to explain how students are grouped into profiles and how adding a clear set of rules helps make these student groups easier for teachers to understand.

Finally, we discuss the current research gap to justify our mixed quantitative and qualitative approach.

2.1 Novice Programmer Behavior

Research in computing education has historically shown the diversity of novice programming behaviors. Two students can obtain similar final scores while following different trajectories: one may iterate quickly through many partial corrections, while another may submit lesser after long pauses and larger rewrites. These differences often reflect distinct regulation strategies, error-recovery patterns, and confidence dynamics.

This observation has direct implications for analytics. If evaluation is reduced to end-state performance, the incremental mechanisms that are part of the learning remain invisible. A process-aware analysis¹ is therefore necessary when the goal is not only to rank students, but also to support them during their progression. In

¹Meaning an analysis focused on the student’s learning journey and problem-solving steps.

this setting, submission traces² provide a useful view on how students regulate[14] their work. Retries after short periods, longer pauses, and occasional larger rewrites can reflect different ways of reacting to difficulty. [9, 4, 7, 13]

2.2 Educational Data Mining in Programming Education

Educational data mining in programming environments generally relies on a combination of logs, test outcomes, and features derivated from codes. This body of work has demonstrated that student behavior can be modeled and, in many cases, predicted with reasonable accuracy. However, a recurring problem appears between predictive power and interpretability. Highly expressive models may improve classification metrics while offering limited explanatory value to instructors.

For this thesis explicitly oriented toward educational use, interpretability is not optional. In this work, model choices are constrained by that requirement: features and profile rules must remain understandable by non-specialist readers.

This is why the thesis favors a smaller set of explicit features and transparent profile rules rather than a more complex predictive setup. [10, 1, 3]

2.3 Temporal Learning Analytics

Temporal learning analytics gives a natural framework for incremental submission data. Inter-submission delays, retry rhythm, and progression regularity reveal the *dynamics* of work, not just its outcomes. In programming education, this temporal dimension is particularly informative because students externalize much of their reasoning through iterative submission behavior.

In practice, timing data is highly irregular and varies from case to case, which requires careful preprocessing and interpretation. However, when combined with performance and effort metrics, they can expose behavioral structure that only final results fail to capture. In this work, time is treated as a behavioral signal. The goal is to understand how students distribute effort across a sequence of tasks and how their rhythm changes when they encounter difficulty. [3, 8]

²This refers to the recorded history of a student’s code attempts and retries over time.

2.4 Behavioral Profiling and Clustering in CS Education

Clustering is widely used to group students based on their programming data³. A common result is finding similar group of students: a high-performing group that keeps trying, and a low-performing group that stops engaging. While these groups are helpful, they are not always easy for teachers to use in a real classroom, especially when the groups are created by data without any clear description of what the students actually did.

To fix this problem, this work combines data clustering with clear rules for profiling where every behavior is determined by four specific features: two features for student performance, and two features for student time. The two parts do different jobs: clustering finds natural patterns in the data, while our four-feature rules give those patterns clear behavioral labels. Checking where the data groups and our rules agree or disagree is a major part of our analysis.

This simple system makes the final groups much easier to discuss with teachers. It gives us a clear vocabulary to talk about common student behaviors based on performance and timing, without losing the benefits of the data-driven grouping itself. [12, 11, 5]

2.4.1 Qualitative Observations of Coding in the Wild

While quantitative studies effectively capture the macroscopic rhythms and sequential patterns of student trajectories, qualitative studies can show a significant difference between functional program behavior and genuine conceptual mastery. In a think-aloud study⁴ examining introductory programming students as they attempted to reverse-engineer a solution from scratch, Kennedy and Kraemer [6] demonstrated that successful task completion does not directly guarantee structural or conceptual understanding [6]. Novices frequently produced and submitted correct code that worked purely by coincidence or through blind, trial-and-error adjustments driven by compiler feedback, all while expressing profound uncertainty regarding their own implementations [6].

These observations on behaviors show the critical limitations of evaluating student progress solely through automated test suites or end-state⁵ functional correctness. It clearly shows the pedagogical necessity of looking deeper into how they incrementally

³We’ve an example with the Dolos tool developed at UGent university for Source code plagiarism detection <https://dolos.ugent.be/>

⁴A think-aloud study is a method where participants speak their thoughts out loud while working on a task, allowing researchers to observe their immediate problem-solving process.

⁵End-state performance refers to evaluating only the final version of the code or the final grade a student achieves, ignoring the history of how they built it.

proceed. It will help to accurately diagnose underlying student reasoning and exhibit fragile knowledge patterns [6].

2.5 Research Gap

The practical gap targeted in this thesis is not the absence of metrics, but the absence of a strong link between three requirements: quantitative pattern discovery, interpretable behavioral characterization, and actionable teaching translation. Existing approaches often satisfy one or two of these requirements, but rarely all three in a single shot.

So, the real problem is not a missing formula or model, but the lack of a clear method that teachers can easily read and use to help their students. [2]

Chapter 3

Context and Data

This chapter describes the data and the environment used in this thesis. The first section introduces the programming course and the automated platform where student data was collected. The next section details the overall dataset composition, including the number of questions, total submissions. Finally, we present an overview of how we process data and a reference to the public github repository for reproducibility facility.

3.1 Course and Platform Context

The empirical setting of this thesis is an introductory python programming course¹ where students submit Python codes through the INGINious² automated platform from UCLouvain. The goal of these programming tasks is to help students practice and learn the computing concepts taught throughout the course. So, this platform context is particularly useful for behavioral analysis because it preserves not only final attempts, but complete submission history and traces. Each attempt usually records a timestamp, scores, and metadata.

The same platform is also used to host the final exam of the course, so this makes it a great database for our analysis.

¹Course pages (same course, different codes by faculty): LINF01101 for Computer Science (<https://uclouvain.be/en-cours-2025-linfo1101>) and LEPL1401 for Engineering (<https://uclouvain.be/en-cours-2025-lepl1401>).

²INGInious autograder documentation: https://docs.inginiuous.org/en/latest/what_is_inginiuous.html.

3.2 Data Overview

This analysis focuses on the january 2026 session dataset containing 588 student records across six programming questions. Each question covers major concepts seen in the course and is designed to evaluate student's level for these specific parts. This dataset was chosen because it represents a complete, consistent and recent set and the questions covers concepts seen across the introductory python programming course.

Question	Total Submissions
q1	10,017
q2	11,940
q3	9,196
q4	8,508
q5	5,891
q6	2,941
Total	48,493

Table 3.1: Dataset Overview: Total Submissions(.py codes files) and Questions

3.3 How the Code Processes the Data

The code for this project works in five main steps to change raw data into finished utilities: cleaning the extracted data, buiding relevant features from these data³, grouping the data into clusters, assigning student profiles and creating visual representations. Each step has its own separate set of scripts and creates its own intermediate files. This way lets us check the data at each stage instead of hiding all the changes inside one large file.

In the GitHub repository (https://github.com/Lord-Melflam/Master_Thesis_Francois_Meli_2026), we show exactly which script creates which output so that it's easy to trace any figure in this thesis back to the exact code that made it.

³Also seen as feature engineering. https://en.wikipedia.org/wiki/Feature_engineering

Chapter 4

Quantitative Results

This chapter presents the quantitative discoveries of our analysis based on the collected data. We first define our methodology, explaining how we use four principal family of features: two for **performance** and two for **timing** that helps us in finding real student habits. Next, we show the global structure of our data, which allow us to split them into two distinct groups: a cluster dominated by bad¹ profiles and another cluster dominated by good² profiles. We validate this choice statistically unsupervised algorithm like the hierarchical clustering, accompanied with elbow and silhouette tests, and we look closely at how student behavior changes question by question. Finally, we conclude by showing interesting and non-intuitive facts we've found in this part of the analysis.

4.1 Methodology: How We Analyzed Student Behavior

Before showing what we found, we'll explain how we analyzed student submissions, from a quantitative aspect.

The goal. We wanted to identify groups of students who behave similarly³: students who work in the same way and progress through problems using similar strategies. To do this fairly, we needed to measure more than just final grades. For instance, a student who gets 60% on the first try is very different from a student who also gets 60% after 50 failed attempts.

¹Profiles dominated by unsatisfactory statistics(in the sense of **performance** and **timing** features' families)

²Profiles dominated by satisfactory statistics(in the sense of **performance** and **timing** features' families)

³Profiling behaviors

What we measured. For each student, we looked at three types of information from their submissions:

- **How well they did:** scores on each submission attempt, and their best score overall
- **How much effort they invested:** number of attempts, how many times they tried again (showing persistence)
- **How they managed their time:** Did they retry quickly after failing, or did they take long breaks? How regular was their work rhythm?

4.2 Finding our optimal number of clusters and profiles

How we grouped students using clustering. We used a statistical method called *hierarchical clustering*, with *Ward*⁴ linkage. We can see that method as a sorting algorithm: we start with 588 individual students, measure how *similar* each pair is (using the three types of informations above), and progressively group the most similar students together. That method is complemented by a tree-like diagram (a dendrogram) showing which students belong together. Results of these groupings are shown in Figure 4.1 and Figure 4.2.

⁴A criterion applied in hierarchical clustering. https://en.wikipedia.org/wiki/Ward%27s_method

Student Behavioral Clustermap (Average features)

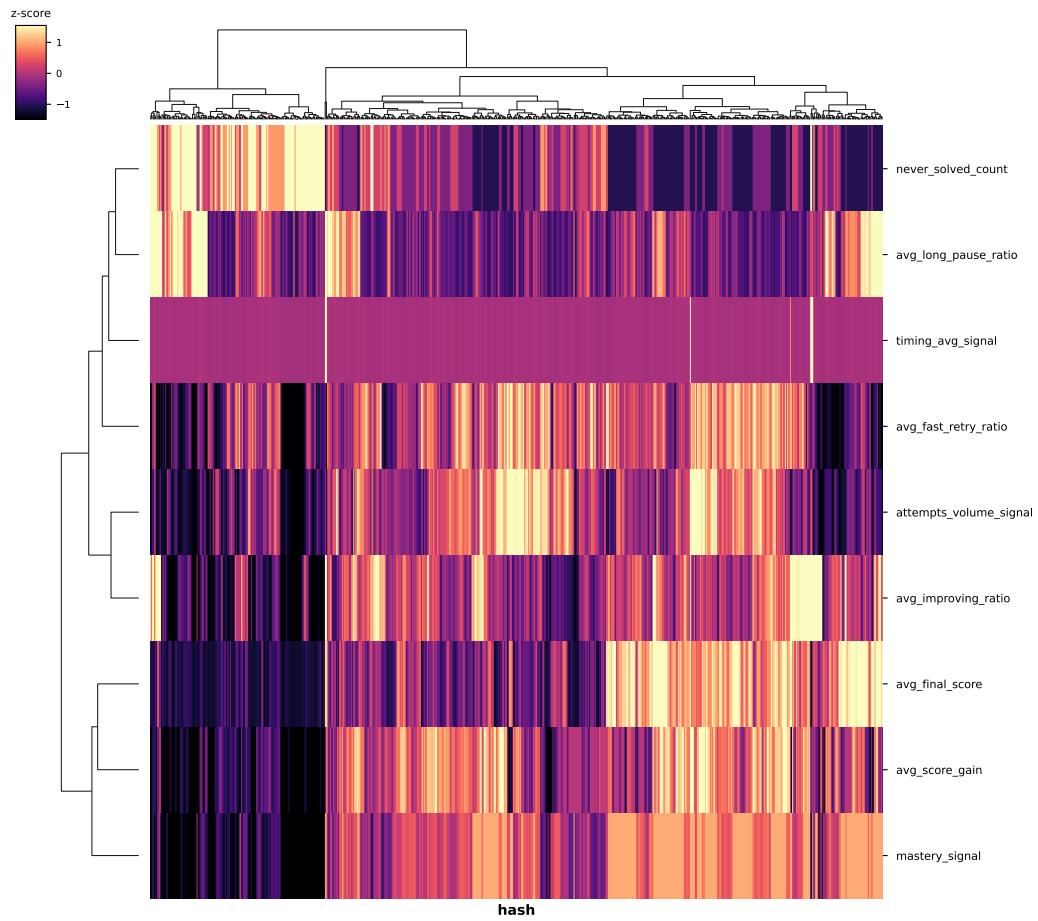


Figure 4.1: Mean-based hierarchical clustermap used for early cluster interpretation.

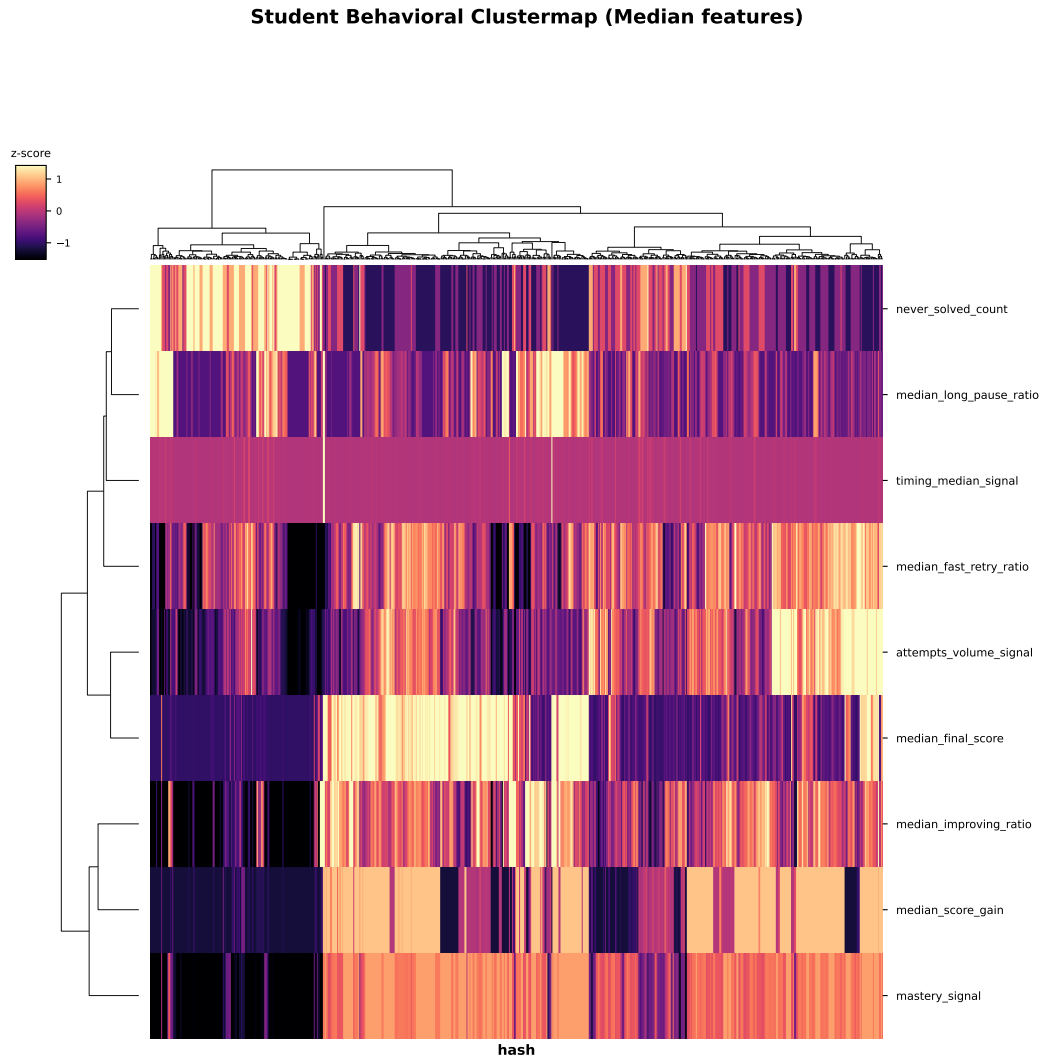


Figure 4.2: Median-based hierarchical clustermap used to evaluate the clustermap stability.

All the engineered features used for that analyzis are detailed in Table D.1 in the Glosssary.

The two clustermaps lead to the same overall partition, which makes the cluster structure easier to trust. The median-based version checks that the result is not biased by a few extreme values.

Deciding how many groups/clusters. The tree can be cut at different levels, creating different numbers of clusters. Figure 4.3 shows the hierarchical dendrogram

structure with horizontal lines marking where to cut for $k = 2$, $k = 3$, and $k = 4$ clusters.

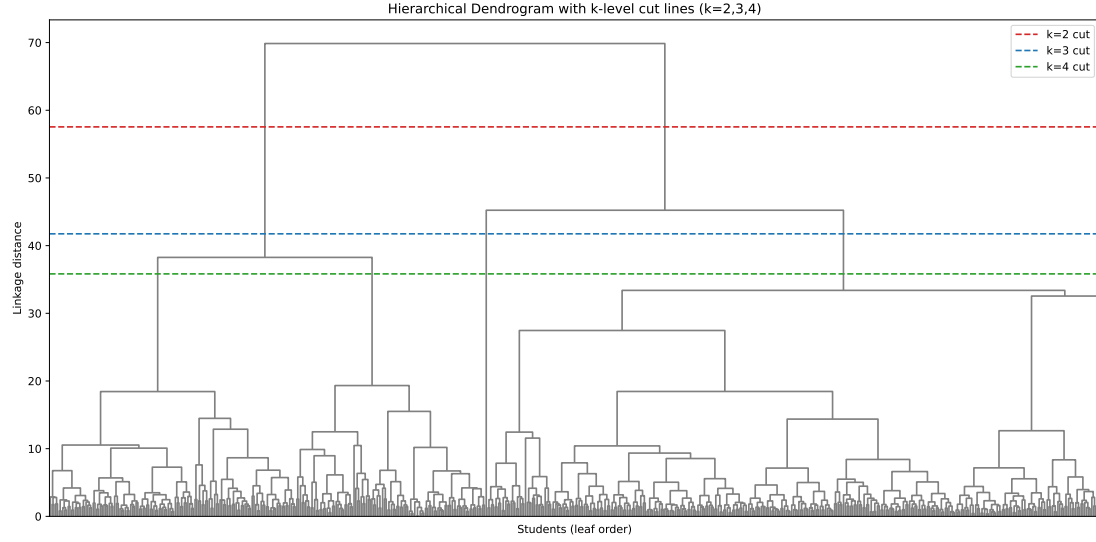


Figure 4.3: Hierarchical dendrogram showing potential cut levels.

In the Figure 4.4, we present what these cuts look like in terms of student's proportions:

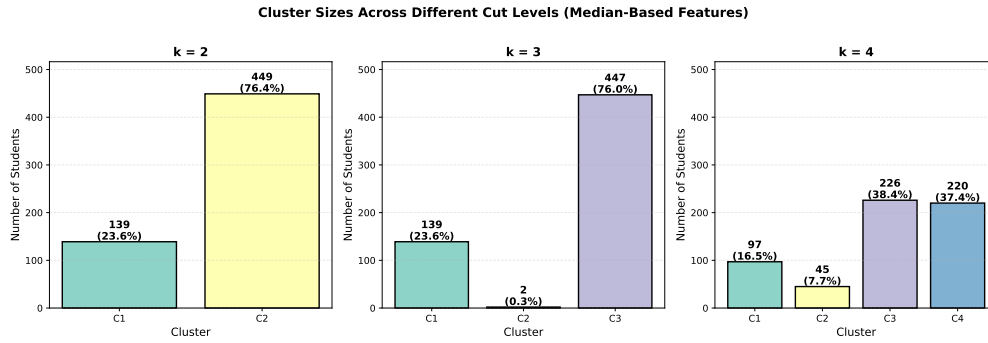


Figure 4.4: Cluster sizes across different cut levels.

At this point of the analysis, a good cut-level⁵, mainly to avoid complicating interpretation without pedagogical gain appeared to be $k = 2$. However, further tests were applied:

⁵The level at which we'll accept the cut from our dendrogram, leading to k clusters

- **Statistical test (Elbow Method⁶):** We measured how much additional information we gain by adding each new cluster.
- **Silhouette Score⁷:** We checked whether students within each cluster are truly similar to each other and truly different from other clusters. Scores near 1.0 meaning tight clusters; near 0 means loose clusters.
- **Pedagogical test:** As a strong validation metric, we asked ourself: "Can a teacher actually use this grouping to understand students and design interventions?"

4.2.1 Elbow method and silhouette score

Figure 4.5 displays the explained variance ratio (elbow curve) and silhouette score(How well clusters are separated) across $k = 2$ to 8 clusters. The explained variance ratio measures the proportion of total variance captured by the clustering.

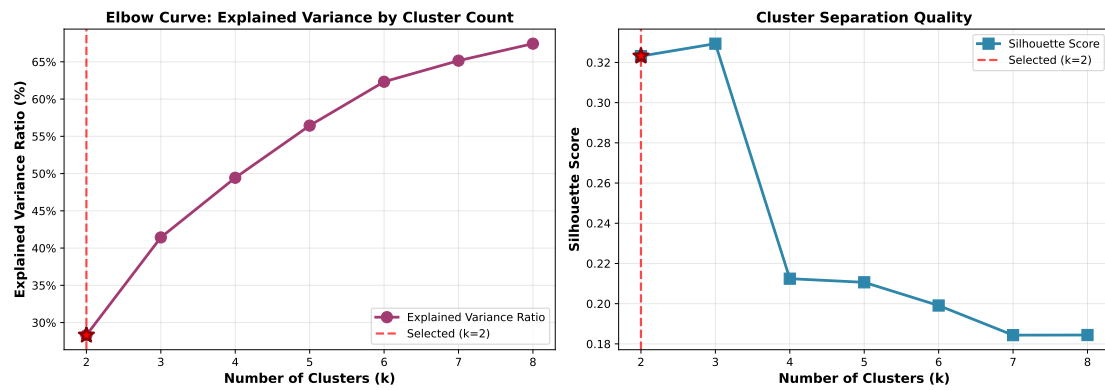


Figure 4.5: Selection of the number of cluster via explained variance ratio and silhouette scores. .

Selection of k based on Slope analysis of the Elbow Method

To avoid arbitrary selection of k , we apply the elbow method using slope analysis. The slope represents the rate of change in explained variance per additional cluster: $\text{slope} = \Delta V / \Delta k$, where ΔV is the change in variance ratio and $\Delta k = 1$ cluster. The elbow point is identified where the slope drops sharply, indicating that adding more clusters doesn't have considerable value.

⁶Heuristic used in determining the number of clusters in a data set. [https://en.wikipedia.org/wiki/Elbow_method_\(clustering\)](https://en.wikipedia.org/wiki/Elbow_method_(clustering))

⁷Method of interpretation and validation of consistency within clusters of data. [https://en.wikipedia.org/wiki/Silhouette_\(clustering\)](https://en.wikipedia.org/wiki/Silhouette_(clustering))

Table 4.1: Slope analysis for cluster selection via explained variance ratio. Calculations show how the variance gain (ΔV , in percentage points) decelerates as k increases, with slope drop percentage indicating the rate at which marginal gains diminish. The largest slope drop occurs at $k = 3 \rightarrow 4$ (-38.9%), marking the statistical elbow point.

Transition	$V(k)$ (%)	ΔV (%)	Slope ($\Delta V/\Delta k$) (%/cluster)	Slope Drop (%)	
$k = 2$	28.31	-	-	-	
$k = 2 \rightarrow 3$	41.43	+13.12	+13.12	-	
$k = 3 \rightarrow 4$	49.44	+8.01	+8.01	-38.9%	← ELBOW
$k = 4 \rightarrow 5$	56.44	+7.00	+7.00	-12.6%	
$k = 5 \rightarrow 6$	62.33	+5.89	+5.89	-15.9%	
$k = 6 \rightarrow 7$	65.15	+2.82	+2.82	-52.1%	

Based on these tests, we still chose $k = 2$ clusters. To justify this choice of two clusters rather than an alternative number, we have to understand that the data reveal a trade-off between statistical metrics and practical utility. In our case, this means:

- **Variance-based elbow:** $k = 3$ is proven to be our elbow point. However, the silhouette score from its side has also something to say.
- **Silhouette score:** $k = 3$ is marginally superior to $k = 2$ (0.3293 vs. 0.3232, a difference of only +0.0061). This small advantage does not clearly justify the added complexity of choosing $k = 3$ over $k = 2$. For $k \geq 4$, silhouette scores deteriorate substantially (0.2124).
- **Pedagogical interpretability:** The two-cluster solution (high-mastery vs. low-mastery) is simpler and directly actionable for instruction, whereas a three-cluster partition would require more complex intervention strategies.

In summary, we retain $k = 2$, even though $k = 3$ is slightly stronger on variance and silhouette metrics. This choice is justified on the aspects listed above. In the Tables 4.2 and 4.3, we present exactly what each cluster looks like, in term of statistics.

Table 4.2: Cluster-level median descriptive summary ($n = 588$).

Metric	Cluster 1	Cluster 2
Students	139	449
Median best score	21.63	84.08
Median attempts	8.42	17.79
Median fast retry ratio	0.423	0.581
Median long pause ratio	0.097	0.059
Median solved rate	0.200	0.833

Table 4.3: Cluster-level mean descriptive summary ($n = 588$).

Metric	Cluster 1	Cluster 2
Students	139	449
Mean best score	22.90	84.20
Mean attempts	9.37	18.71
Fast retry ratio	0.390	0.565
Long pause ratio	0.138	0.083
Solved rate	0.187	0.807

4.2.2 Proportion of profiles accross $k = 2$ clusters

The four-dimensional profile vocabulary. To make student behavior easier to talk about and act on in teaching, we use four yes/no questions:

1. **Persistence:** Do they keep trying? (high or low attempt count⁸)
2. **Effectiveness:** Do they succeed? (high or low best score⁹)
3. **Iteration quality:** Do they retry quickly? (high or low fast-retry ratio¹⁰)
4. **Engagement:** Do they submit regularly? (low or high long-pause ratio¹¹)

These four dimensions create $2^4 = 16$ possible types of profile. Table D.2 gives detailed explanations about these profiles. Each student can be classified into one, based on where their measurements fall relative to median¹² thresholds for the dataset.

⁸Based on `avg_attempts / median_attempts` threshold

⁹Based on `avg_best_score / median_best_score` threshold

¹⁰Based on `avg_fast_retry_ratio / median_fast_retry_ratio` threshold

¹¹Based on `avg_long_pause_ratio / median_long_pause_ratio` threshold

¹²Naturally, median based features appeared to be less sensitive than the mean based ones.

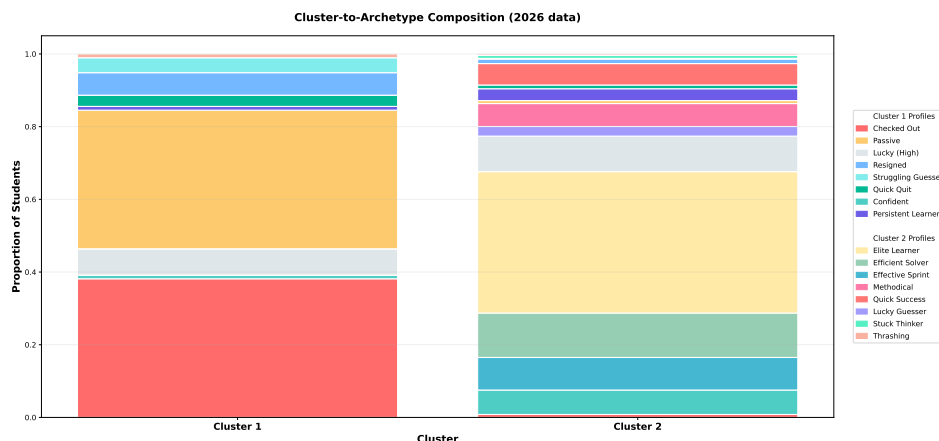


Figure 4.6: Profile Distribution within Each Cluster

At the end, we cut the hierarchical results at the level that best separates students into two clusters: This cut produces Cluster 1 with 139 students (24%) and Cluster 2 with 449 students (76%). The colored rectangles in Figure 4.6 show which profile type each student belongs to, with Cluster 1 having a higher proportion of *Checked Out*, *Passive* or *resigned learners* showing low-mastery¹³ behavior., and Cluster 2 having satisfactory profiles such as *Elite learner* and *Efficient Solver* showing high-mastery¹⁴ behavior. Each student profile archetype¹⁵ is assigned a distinct color for visual clarity.

The next sections will be about deeper analysis into what else the data used in this thesis, can teach us.

4.3 Further observations

We start this section by demystifying a belief that many students might have while learning introductory programming course.

4.3.1 Demystifying a myth: "Question difficulty increases the higher the question index is"

Many students believe: "If I cannot solve Question 1, I will surely fail Question 6." Testing this with failure correlations reveals a different story. Figure 4.7 shows that Q1-Q6 failure correlation is only $r = 0.234$ (weak), meaning failure on Question 1

¹³Relatively good coding mastery, related here to "good" stats

¹⁴Relatively good coding mastery, related here to "good" stats

¹⁵Synonym of "pattern"

does not strongly predict failure on Q6. However, Q5-Q6 correlation is $r = 0.402$ (stronger), suggesting another factor might influence at some point in time. In fact, because student tends to do Question 6, generally later in the exam(for those reaching that part), cumulative fatigue can matter more than the initial difficulty. According to the teachers themselves, later questions are even sometimes easier to solve than their predecessors.

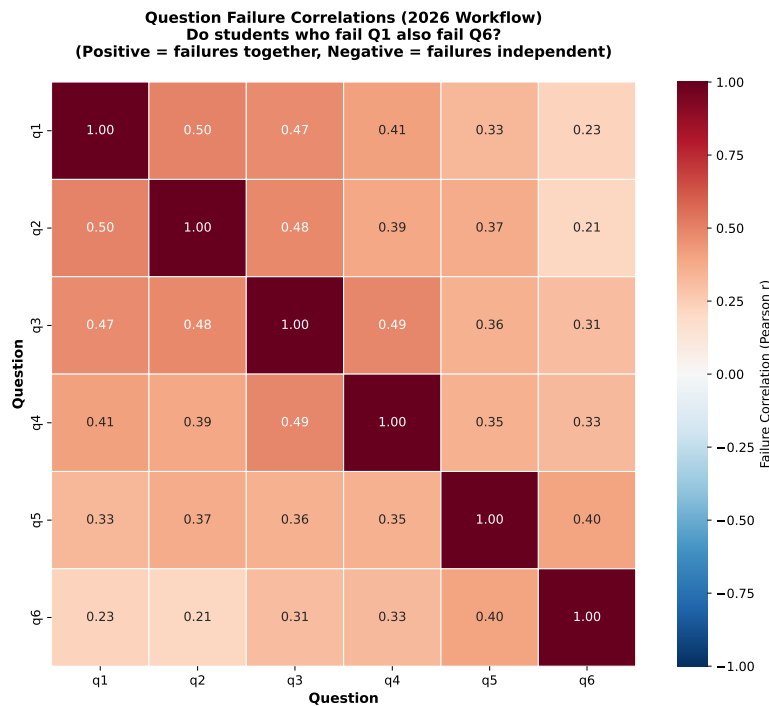


Figure 4.7: Failure correlations across questions. The weak Q1-Q6 link ($r=0.234$) demystifies the linear difficulty assumption. The strong Q5-Q6 link ($r=0.402$) points to cumulative fatigue as the potential main constraint in the final questions.

Table 5.2 from Chapter 5 presents the different question topics from the january 2026 session.

4.3.2 Submission Trajectories

The family tree style dendrogram gives us a great big-picture view, but it hides the step-by-step order of how students actually moved through the questions. To see that progression, Figure 4.8 tracks the exact relationship between the time spent (in hours) and the final score achieved by each individual student across all six questions.

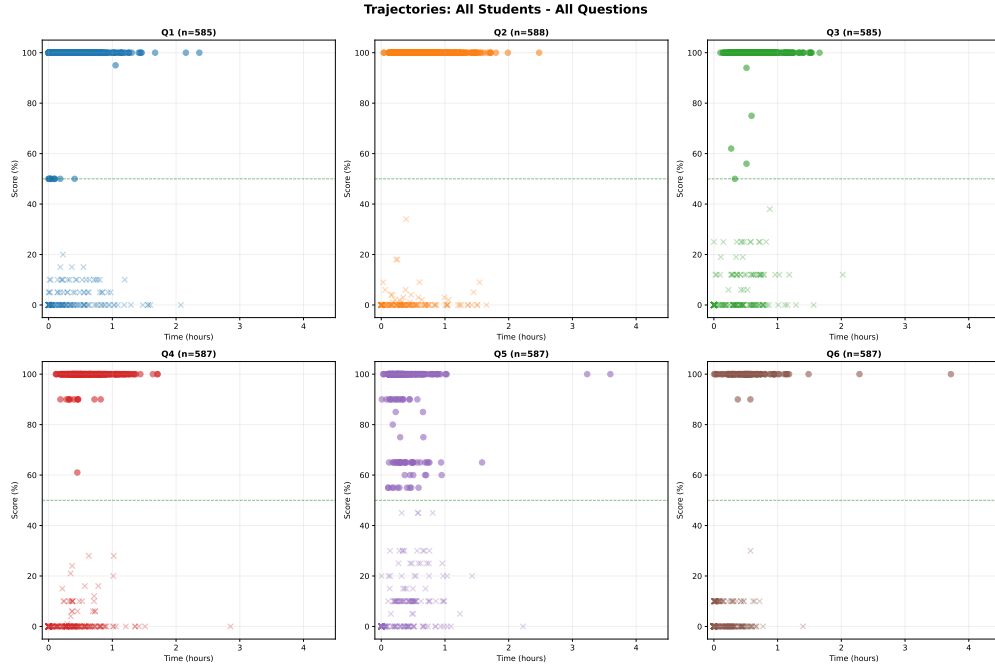


Figure 4.8: Individual time versus score distributions across all six exam questions. Circular markers at the top represent students who passed the problem ($\geq 50\%$), while cross marks (X) represent a failing outcome ($< 50\%$). The dashed horizontal line highlights the passing threshold.

The distribution of points on Figure 4.8 show three distinct aspects:

- On Q_1 , Q_2 , there is a dense concentration of points clustered at 100% within the first 30 to 45 minutes, showing a large group of students who reached full marks quickly.
- On Q_3 , Q_4 and Q_5 , the points spread out horizontally and behind 1 hour (except for Q_5). On Q_2 , many of these long-duration points still reach 100%, whereas on Q_3 and Q_4 , the points past 2 hours are exclusively failures (X) concentrated below 20%.
- On Q_6 , the distribution changes to a distinct split between failures (X) and success.

4.4 Grades, Speed, and Persistence Across the Clusters

Looking closely at the student groups we found, we can see important differences in their final grades, how fast they retried questions, and when they decided to give up.

4.4.1 Final Scores and Success Rates

Students in the high-mastery group did much better than those in the low-mastery group. The average best score shows a huge gap (around 84.2 points versus 22.9 points), which matches the percentage difference in how many questions they actually solved (0.81(81%) versus 0.19(19%)). These differences show two completely different levels of understanding and tells us that these two groups of students need completely different kinds of help or teaching strategies.

4.4.2 How Students Pace Their Attempts

Students who did well tended to submit their code quickly and took fewer long breaks. High-mastery students had a much higher fast-retry rate (≈ 0.565 versus ≈ 0.390) and a lower long-pause rate (≈ 0.083 versus ≈ 0.138). This shows that successful students submitted code, and fixed their mistakes in a consistent way. On the other hand, struggling students took long pauses between attempts, meaning they either got stuck or abandoned early.

4.5 How Questions Link Together

Analyzing the questions shows that the **success** of a student on one problem usually shared a moderate correlation with how they did on the next one, as shown in Figure 4.9. The strongest links happened between questions that were right next to each other in the exam, like (q_1, q_2) , (q_3, q_4) , and (q_4, q_6)

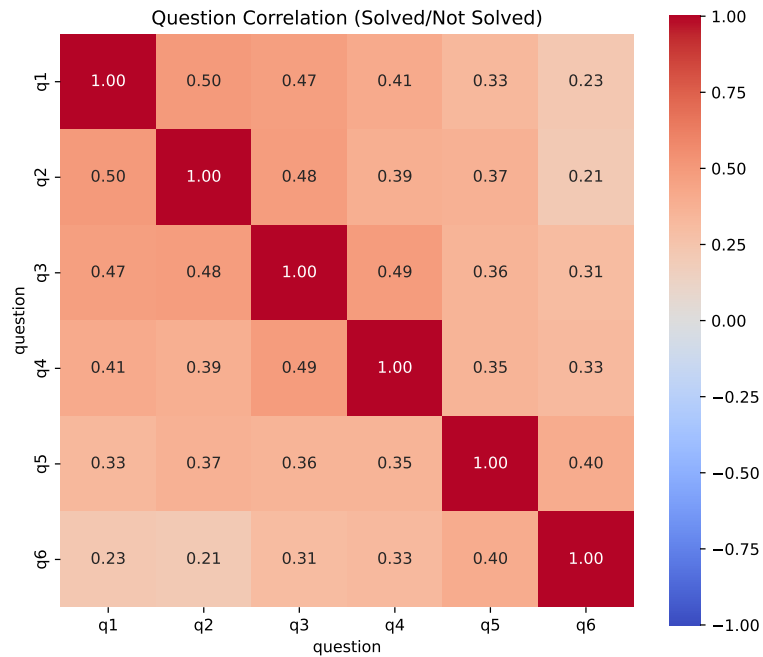


Figure 4.9: Correlation matrix showing how success on one question links to success on another.

4.6 Students Stuck in the Middle

Even though the two main groups are clearly split, there're few profiles groups that ended up right on the borderline between them. These cases are visible inside the Figure 4.10 and can be looked more significantly inside the Figure 4.6. Example with the *Persistent Learner* or the *Lucky (High)* profiles.

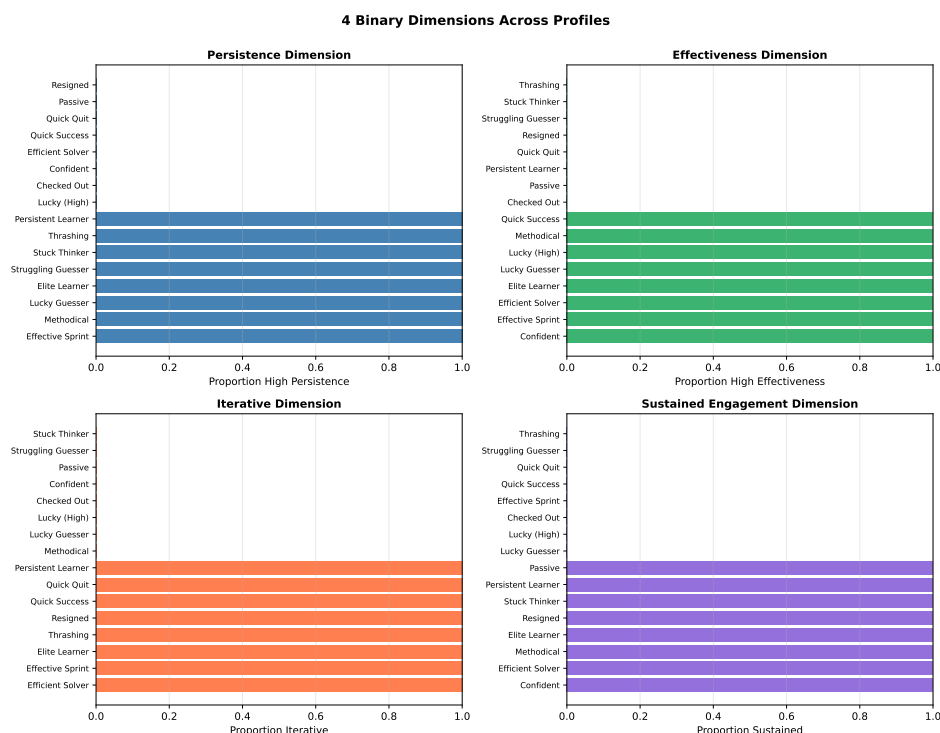


Figure 4.10: How different behaviors are spread out, showing which habits split the groups clearly and which ones blur together.

4.7 Time Analysis: How Students Split Their Time and Energy

Looking at how much time students spent on each question gives us a clear view into how hard they found the problems and when their performances started degrading during the exam.

4.7.1 Time Spent on Each Question

We calculated the active time a student spent on a question by measuring the gaps between their consecutive code submissions. Figure 4.11 shows this result, after filtering out 9 extreme anomalies (gaps longer than 24 hours) to keep things realistic for the standard 3-4 hour exam session¹⁶.

¹⁶Students with special needs often get more exam time than others.

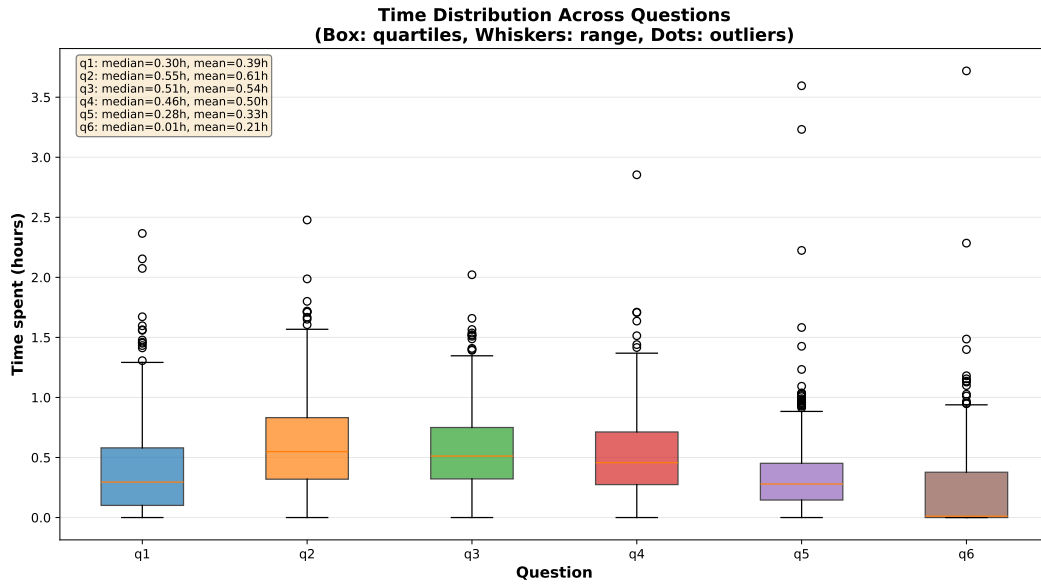


Figure 4.11: Spread of active time spent on each question across the 588 students. The boxplots show the medians, middle part, and individual outliers.

We see strong effort on Q_2 (average: 0.61 hours, median: 0.55 hours) and it stays stable all the way through Q_4 . But time drops on the last two questions, falling to an average of just 0.33 hours on Q_5 and 0.21 hours on Q_6 . Summing up the average times gives a total of ≈ 2.58 hours, which fits the actual exam window and shows a transition from stable, hard work (Q_1 - Q_4) to potential rushed, last-minute attempts (Q_5 - Q_6).

4.7.2 Does Spending More Time Mean Better Grades?

To see if working longer actually helped students pass, Figure 4.12 maps time spent directly against final scores.

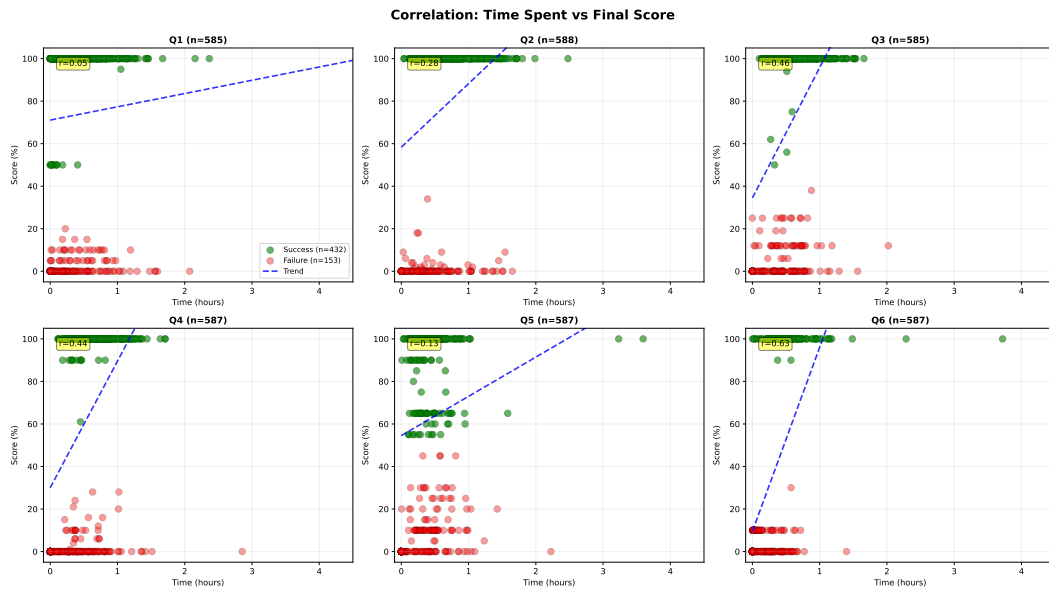


Figure 4.12: Graphs comparing time spent versus final scores for all six questions. Green dots mean the student passed ($\geq 50\%$); red dots mean they failed.

Table 4.4 summarizes these tendencies along with the actual pass rates for each question.

Table 4.4: Time-score correlation values and pass rates for each question.

Question	Correlation (r)	Pass rate	What it shows
Q1	0.05	73.8%	High pass rate: Time spent didn't matter; most passed regardless.
Q2	0.28	76.2%	Constant engagement: A slight link where extra time helped a bit.
Q3	0.46	66.3%	Moderate link: putting in more time clearly paid off.
Q4	0.44	59.3%	Moderate link: persistence was key to get good scores.
Q5	0.13	61.8%	Transition phase: where students probably started getting tired.
Q6	0.63	26.4%	Strongest link: saving enough time was important to pass.

4.7.3 Exam Progression and the Impact of Fatigue or External Factors

Figure 4.13 tracks how average time spent and average scores varied as students moved through the exam.

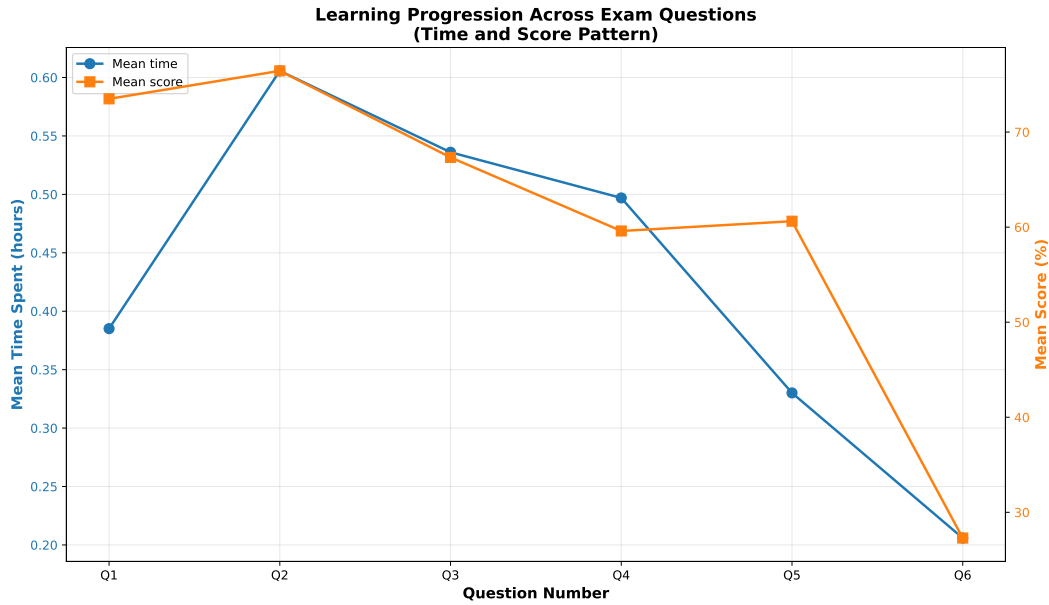


Figure 4.13: Progression curve, mapping average active time (blue, left axis) against average score (orange, right axis) from Question 1 to Question 6.

From Figure 4.13, we can separate the observations into 3 distinct phases in the exam experience:

1. **Questions Q_1 - Q_2 :** Time spent goes up while scores stay high.
2. **Questions Q_3 - Q_4 :** Time remained constant at around half an hour per question, but scores start dropping steadily (67% down to 59%), showing that the problems were getting harder.
3. **Questions Q_5 - Q_6 :** Time spent is nearly cut in half (0.33 down to 0.21 hours) as final grades collapse (61% down to 26%). Students either completely ran out of energy, or they made a strategic choice to stop working on the final question so they could spend their last few minutes double-checking their answers on the earlier problems.

4.8 Quantitative Summary

This analysis has identified two distinct groups of students based on their overall similar patterns across all coding exercises. The high-mastery cluster contains students who typically complete problems with fewer attempts, faster recovery after failures, and less time spent paused between work sessions. The low-mastery

cluster contains students who engage with more attempts, longer breaks, and slower retry patterns, but still achieve lower outcomes.

However, an important note is that **student's behavior is not constant**. In fact, we've observed regularly, for a same student to perform differently accross different exercises. We believe at this point that the main cause can be depending on a non exhaustive list of factors such as question difficulty, their background with the topic, fatigue or their mood that day. For example, A *Persistent Learner* on one exercise might behave differently on another exercise. A *Struggling Guesser* on one question might find a different question easier and show characteristics related to a ddifferent profile. These variations has been taken into account.

Concretely, to take into account this variability, profiles assignment was performed on aggregated features across all six exercises. Rather than treating each exercise separately, we compute a single summary profile per student that captures their overall profile behavior¹⁷. This approach is meant to identify the stable behavioral pattern that characterizes each student across a whole session.

Formally, let q_i represent the vector profile of student X for an individual question i (where $i \in \{1, 2, \dots, n\}$). The global behavioral profile Y_X for that student is defined as the median aggregation across all exercises:

$$Y_X = \text{median}(\{y_1, y_2, \dots, y_n\}) \quad (4.1)$$

¹⁷We can see that in a sense that, Student X has profile Y, means that Profile Y is the aggregation of question-level profiles observed for the same student X

Chapter 5

Qualitative Results

While the quantitative profiles from the previous chapters show *what* the students did, numbers alone cannot explain the logic behind their actions. This chapter looks at the actual code submissions to see exactly where students got stuck and how they tried to fix their mistakes. This helps us understand the real strategies and human thoughts behind each behavioral profile. First, we present how we selected these cases. Then, we show one clear code example for each profile. Finally, we use these observations to directly answer Research Question 2 (**RQ2**) and explain why students adopt these behaviors.

5.1 How the cases were chosen

We selected and analyzed 23 detailed cases across 6 student behavioral profiles, as summarized in Table 5.1. Those profiles are good candidates accross different clusters, prior to our quantitative analysis.

Profile	Number of Cases
Elite Learner	4
Quick Success	4
Persistent Learner	4
Checked Out	4
Struggling Guesser	4
Thrashing	3

Table 5.1: Distribution of Cases Across Profiles

Each case documentation includes the target question, a truncated student identifier hash, main evaluation metrics, and a code snippet. To preserve a decent report structure and narrative clarity, this section presents for each selected profile,

one of the most relevant example. The remaining 17 can be accessible via the github¹ repository.

Q	Exam topic	Question goal
Q1	Matrix transposition	The student function is <code>transpose(image)</code> . The question requires returning a new square binary matrix transposed across the main diagonal, without modifying the input.
Q2	Conway	The function <code>conway(lst)</code> must compress runs of equal values into <code>[count, value]</code> pairs, with randomized and deterministic checks for sequence continuation.
Q3	Table allocation / reservation matching	The function <code>affecter_tables(tables, reservations)</code> requires assigning each reservation to the smallest available table that can fit it, using first-fit behavior among admissible tables.
Q4	File parsing and longest word extraction	The function <code>longest_word(filename)</code> requires returning a 3-tuple <code>(line, index, word)</code> for the longest word in a text file and must raise <code>FileNotFoundError</code> if the file is missing.
Q5	Object-oriented inheritance: employee/commercial class	The question requires a <code>Commercial</code> class inheriting from <code>Employee</code> , with a proper constructor, a <code>bonus()</code> method, and a redefined <code>salaire()</code> that combines base salary and commission.
Q6	Weighted average with linked lists	The function <code>moyenne_ponderee(student_list, course_list)</code> has to compute a weighted average using linked lists, returning the correct numeric result without mutating the lists.

Table 5.2: Topics of the January 2026 exam questions.

5.1.1 What we look for in the code

The qualitative review focuses systematically on specific indicators within the submissions:

- Logical structure versus, random rewrites.
- Correct implementation of loops, index boundaries, and conditional branches.
- Whether successive retries demonstrate progressive logical adjustments.

¹https://github.com/Lord-Melflam/Master_Thesis_Francois_Meli_2026

- Visible signs of active problem-solving approach, hesitation, or complete abandonment.

5.2 Representative Student profiles

5.2.1 Elite Learner

Case 1: bc2d4163fa374ff3e...

Question: q3 (Table assignment)

Metrics: 15.33 attempts; 100.0% best score; fast retry ratio 0.707; long pause ratio 0.021

What we see: Strong mastery pattern. The data dependencies are clearly mapped, and edge cases are evaluated cleanly from the start + good usage of data structures.

Code snippet

```
def affecter_tables(tables, reservations):
    d = {}
    for i in reservations:
        if len(tables) == 0:
            d[i[0]] = None
        pos = []
        for t in tables:
            if t[1] >= i[1]:
                pos.append(t)
        if len(pos) == 0:
            d[i[0]] = None
        else:
            ...
```

5.2.2 Quick Success

Case 5: 9bedb4d7c4693191d...

Question: q6 (Weighted average)

Metrics: 7.17 attempts; 100.0% best score; fast retry ratio 0.518; long pause ratio 0.173

What we see: Highly efficient navigation. The solution requires minimal adjustments because the initial implementation is conceptually accurate.

Code snippet

```
def moyenne_ponderee(notes_etudiant, credits_cours):
    num = 0
    deno = 0
    liste_note = []
    prev = notes_etudiant.head
    for note in range(notes_etudiant.length):
        liste_note.append(prev.cargo)
        prev = prev.next
    liste_credit = []
    prev = credits_cours.head
    for note in range(credits_cours.length):
        liste_credit.append(prev.cargo)
```

5.2.3 Persistent Learner

Case 10: a96e8f71351fbe705...

Question: q1 (Transpose matrix)

Metrics: 22.60 attempts; 45.4% best score; fast retry ratio 0.581; long pause ratio 0.073

What we see: High engagement but issues in the code structure. The loop mechanics are present, but the student repeats index complexifying the problem without resolving the previous evaluation error.

Code snippet

```
def transpose(image):
    n = len(image)
    r = [0] * n
    for i in range (n):
        for j in range (n):
            r[i] = image[j-i]
    return r
```

5.2.4 Checked Out

Case 13: 63b06bcae92778b37...

Question: q4 (Longest word in file)

Metrics: 1.33 attempts; 0.0% best score; fast retry ratio 0.000; long pause ratio 0.167

What we see: Minimal, unaligned submission syntax, auxiliary function (def s(ligne,n_ieme,mot):) without real necessity. The logic relies on

generic user inputs and raw print actions rather than conforming to the expected functional design, leading to quick abandonment.

Code snippet

```
def s(ligne,n_ieme,mot):
    return ligne, n_ieme, mot
def longest_word(nom_fichier):
    n_f = input("entrer le nom du fichier: ")
    for s in nom_fichier:
        for i in s:
            return s
    nom_fichier= open(n_f, "r")
    print(nom_fichier.read())
```

5.2.5 Struggling Guesser

Case 17: 7451bdfd2ede2781a...

Question: q3 (Table assignment)

Metrics: 15.00 attempts; 12.67% best score; fast retry ratio 0.307; long pause ratio 0.189

What we see: Unstructured iteration. The nested loops and index conditions are modified blindly across entries without a coherent algorithmic plan.

Code snippet

```
def affecter_tables(tables, reservations):
    d = {}
    i = 0
    j = 0
    for elem in reservations:
        for table in tables:
            while i < len(reservations) and j < len(tables):
                if reservations[i][1] <= tables[j][1]:
                    d[reservations[i][0]] = tables[j][0]
                    i += 1
                    j += 1
            else:
```

5.2.6 Thrashing

Case 22: 8bf6733f987866aad...

Question: q3 (Table assignment)

Metrics: 22.50 attempts; 40.0% best score; fast retry ratio 0.627; long pause ratio 0.144

What we see: High-frequency, random modifications. The student is frequently trying to hard-code static parameters to clear isolated error flags.

Code snippet

```
def affecter_tables(tables, reservations):
    table= []
    t = (21, 2)
    tables.append(t)
    reservation = []
    t2 = ("Arley", 5)
    reservation.append(t2)
    affecter_table ={}
    for i, (a, b) in enumerate(zip(table,reservation)):
        if b <= b:
            affecter_table[table[a]] = reservation[a]
    return affecter_table
```

5.3 Answering RQ2: Why Students Adopt These Behaviors

By mapping these specific code errors directly back to the execution metrics, we can move beyond simply categorizing student output and address *why* these distinct behavioral patterns occur. The qualitative patterns reveal four primary underlying drivers behind student decision-making:

5.3.1 Extreme reliance on the Autograder as an Interpreter

A main cause of high-frequency submission patterns, characteristic of the *Thrashing* and *Struggling Guesser* profiles is using the autograder platform as a live interpreter rather than a final verification tool². Instead of verifying local variables or running manual desk checks, students perform trial-and-error adjustments directly inside the browser. In **Case 22**, for instance, the student hard-coded a specific expected

²It can make sense as we know that students for this specific course don't have access to local interpreter, so they might be tempted to submit more frequently to keep their code syntactically correct

tuple result (`t2 = ("Arley", 5)`) directly into the function block. This proves the student is coding reactively to patch whichever error string the autograder returned on the previous run, rather than developing a proper code structure.

5.3.2 Getting Stuck on the Same Logic

The *Persistent Learner* profile includes students who work hard and submit many times (often 20 or 30 times, like in **Case 10**) but never get a perfect score. The code shows that these students get stuck on one specific idea. In **Case 10**, the student wrote the correct outer loops, but kept trying different variations of a broken index formula (`image[j-i]`). Maybe because the Ingenious autograder feedback is not adequate? Anyway, the student gets trapped. They change the exact same line of code over and over, completely missing the fact that their main plan is wrong.

5.3.3 Giving Up Early to Save Time

The low attempt counts and unfinished code in the *Checked Out* profile (like **Case 13**) happen because students make quick choices during a timed exam. When faced with a hard question or a topic they do not know well (like **file handling** in Q_4), some students quickly decide to stop working on it. In **Case 13**, the student just wrote basic input and print statements that did not match the required function format. When their first try failed the autograder check, they simply abandoned the question. They did this to save their remaining time and energy for simpler problems where they knew they could get points.

5.3.4 Having a Clear Plan From the Start

On the other hand, students in the *Elite Learner* and *Quick Success* profiles behave differently because they only use the autograder to check a solution they have already thought through. As shown in **Case 1**, their code is organized and clean from the very first submission. They check for edge cases early (`if len(tables) == 0`) and the logic is easy to read. These students rarely have high submission counts because they do all their problem-solving and thinking before typing their code into the browser. They might use Ingenious for debugging purpose, but the main goal is to confirm their plan works, not to figure out how to write it from scratch.

Chapter 6

Survey Results and Teaching Tips

This chapter connects our survey answers with the previous chapters. The goal here is to see what these combined results mean for real teaching. We show how the different data sources¹ fit together, present exactly where students get stuck, and give simple ways for teachers to help different types of students.

6.1 Survey Results: Where and why Students Get Stuck

To find out where and why students get stuck, we sent out a simple Google Form² survey to test how well they understand code and what are their impressions on specific parts of our pilot course³.

6.1.1 Survey Design

The survey was made to contain multiples sections. Each section assessing different informations like the respondant profile (Bachelor 1, ...) as our main target was bachelor 1 students(from UCLouvain). We also assessed the type of behaviors while coding, the difficulty of each code related part according to the respondant. The section measuring coding skill was section D. It showed students six short Python code examples (P1 to P6). Students did not have to write code. They just had to

¹We have 2 sources: The first one is the data coming from the Ingenious autograder and the seconde one is the data collected during the survey.

²form FR: <https://forms.gle/yYkXQxw84CocJBB78> - form EN: <https://forms.gle/N3YQajguGCnVEH5R6>

³UCLouvain python introductory course. LINFO1101 for Computer Science (<https://uclouvain.be/en-cours-2025-linfo1101>) and LEPL1401 for Engineering (<https://uclouvain.be/en-cours-2025-lepl1401>)

read the pre-filled text, guess what the code would print, and rate how hard it was from 1 (very easy) to 5 (very hard).

Table 6.1: The six pre-filled code questions in the survey.

Question	Topic	What it tests
P1	Basics	Easy math and simple variables.
P2	Loops	How loops count and where they stop.
P3	Functions	Passing arguments and returning values.
P4	Lists & Search	Finding an item inside a list.
P5	Files	Reading text from a file.
P6	Classes	Simple Object-Oriented Programming and inheritance.

Figure 6.1 shows the distribution of respondent based on their academic level for this survey.

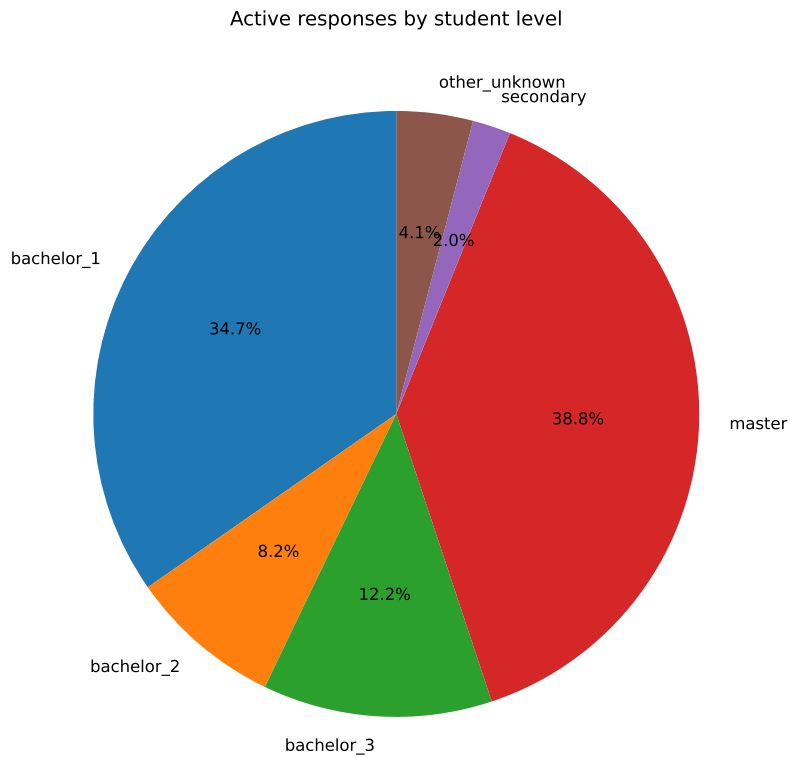


Figure 6.1: Student levels for the survey answers.

6.1.2 The Main Weak Spot: Understanding Loops

We collected answers from 49 students. The results show a clear drop in scores for one specific topic (**Loops and algorithms**).

Table 6.2: How many students got each question right .

Question Topic	Correct Answers	Accuracy
P1 (Basics)	46 / 49	93.9%
P2 (Loops)	24 / 49	49.0%
P3 (Functions)	46 / 49	93.9%
P4 (Lists & Search)	38 / 49	77.6%
P5 (Files)	37 / 49	75.5%
P6 (Classes)	42 / 49	85.7%

While basic code and classes had high success rates (over 85%), only 49% of the students could guess the correct output for the loop question (P2). This showed that in that specific context, loops are the biggest conceptual issue for these students. Figure 6.2 shows self assessment of respondent on the level of difficulty of the 6 small coding tests.

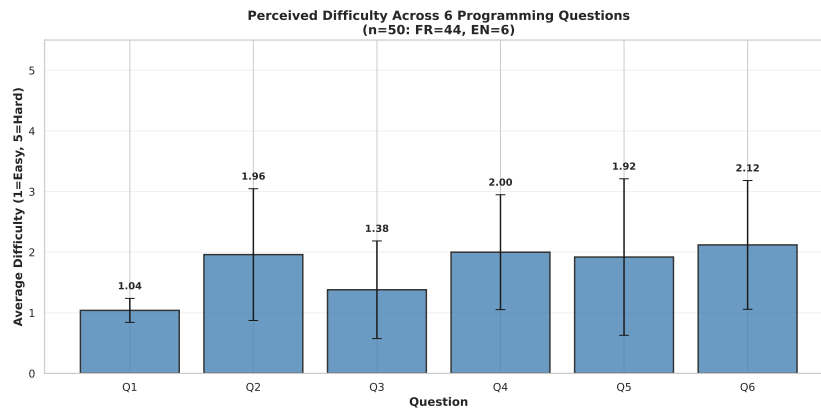


Figure 6.2: How hard students said each question was (from 1 to 5).

6.1.3 First-Year Students Struggle the Most

When we separate first-year students (Bachelor 1) from Master students, we see that the new students have a much harder time.

Table 6.3: Accuracy scores compared between Bachelor 1 and Master students.

Question Topic	Bachelor 1 Accuracy	Master Accuracy
P1 (Basics)	100.0%	100.0%
P2 (Loops)	47.1%	57.9%
P3 (Functions)	94.1%	100.0%
P4 (Lists & Search)	76.5%	78.9%
P5 (Files)	58.8%	84.2%
P6 (Classes)	82.4%	100.0%

Table 6.3 shows that bachelor 1 students have low scores on **loops** (47.1%) and **file reading** (58.8%). This tells us that teachers need to focus heavily on these two topics early in the course.

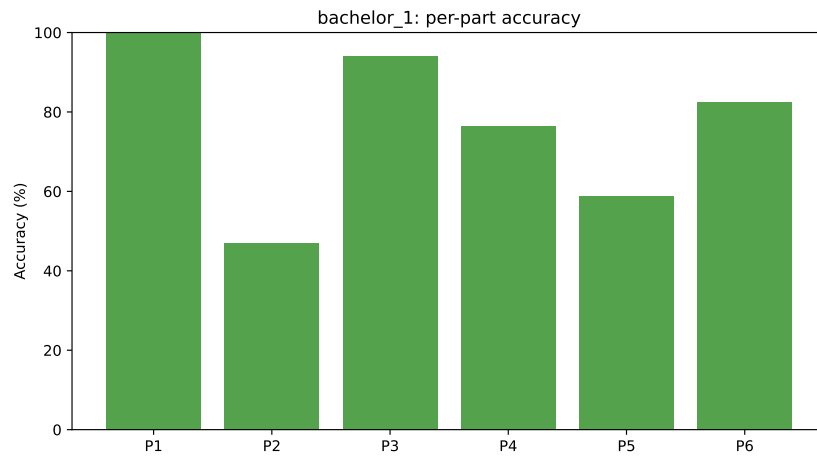


Figure 6.3: Question scores for Bachelor 1 students only (Loops are the lowest).

Also, when we asked all the respondent what they do when they get stuck, Figure 6.4 represent their responses.

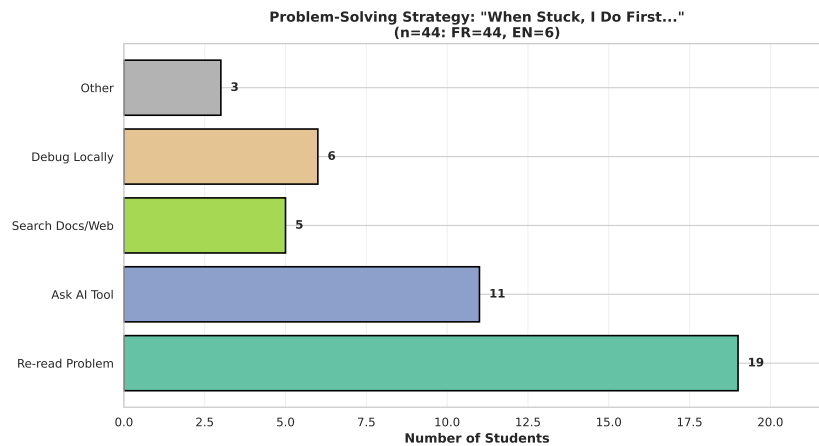


Figure 6.4: What students actually do when they get stuck on a problem.

In the other hand, when we asked all the respondent why they found some specific coding part difficults, the Figure 6.5 shows their responses.

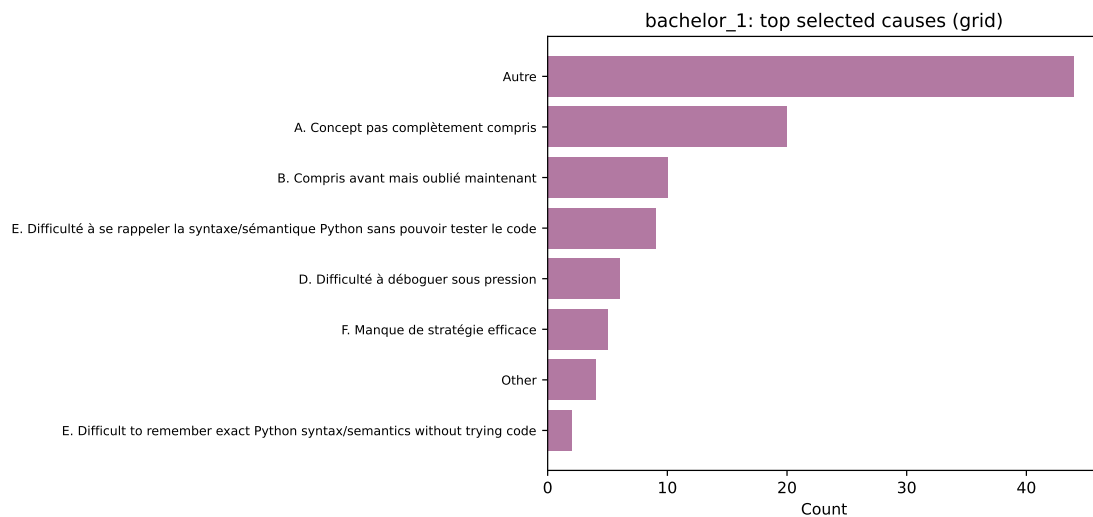


Figure 6.5: What respondent select as explanation for the fact that they found some codes more difficult than other.

We can also find a difficulty heatmap in the Figure 6.6, for further vizualisation. It shows which topics are the hardest for each student.

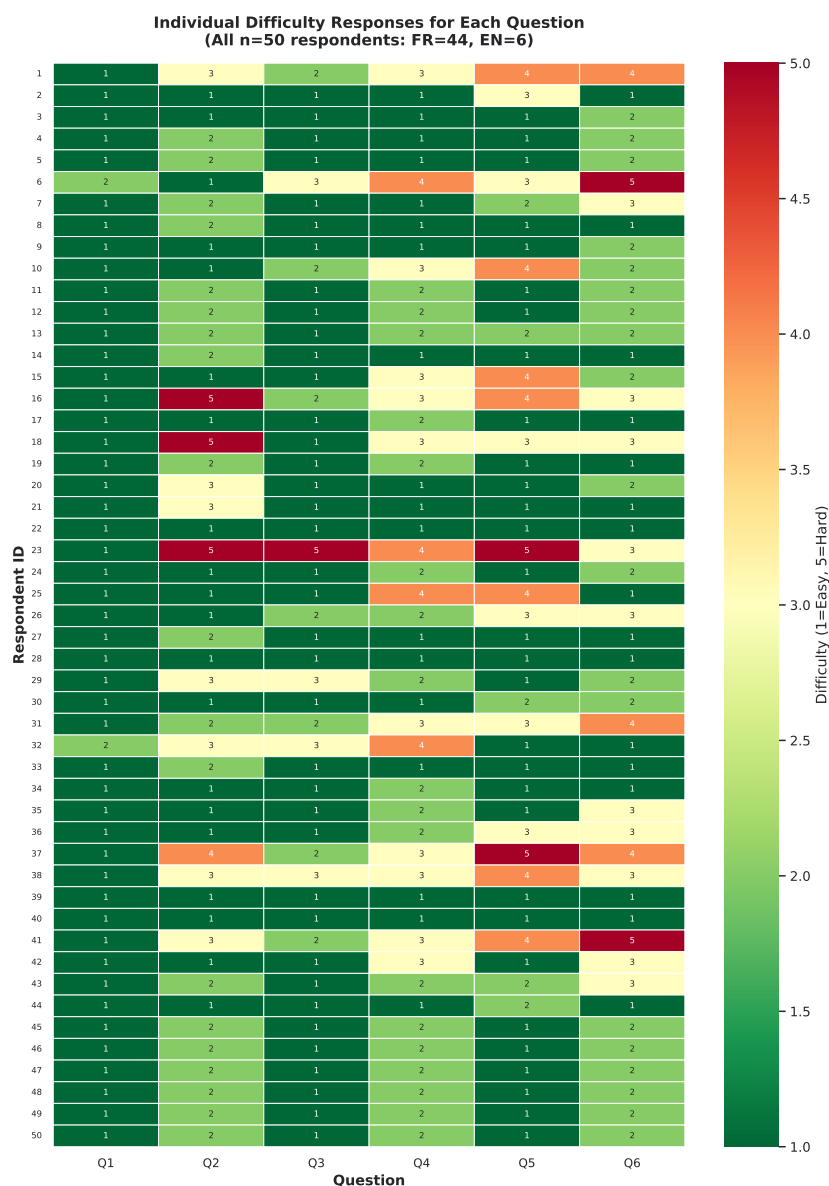


Figure 6.6: Heatmap showing which topics are the hardest for each student.

6.2 Matching Student Profiles to Teaching Advice

The main lesson from our results is that, how a student works shows how well they understand the course materials. Students who take breaks and think between attempts do well. Students who change code fast without taking pauses stay stuck

more often.

Because different students have different problems, giving the exact same help to everyone does not work. Instead, teachers should change their help based on the student's behavior style. The Table 6.4, present some suggested intervention for some profiles:

Table 6.4: How to help different types of students based on their profiles.

Student Profile	What the profile Show	How Teachers Can Help
Elite Learner / Quick Success	Few tries, long pauses, good scores.	Give them harder exercises or let them help other students.
Thrashing / Guesser	Many tries, very fast clicks, bad scores.	Stop them from coding and make them explain their errors as in the think-aloud study[6].
Persistent Learner	Many tries, slow progress, small score fixes.	Help them step-by-step with index tracking and variables.
Checked Out	One or two tries, then they disappear.	Send them a friendly message to check if they are lost.

6.3 Chapter Summary

This chapter combined our survey answers, quantitative and qualitative analysis to create clear tips for teachers. The principal result is that understanding course materials concepts such as *loops*, *algorithms*, *files operations* and learning how to self assess their mistakes are the two main struggles for new programmers. However, instead of treating every student the same way, teachers can get better results by matching their support to the specific behavior styles of those.

Chapter 7

Conclusion

This thesis looked at how novice programmers work, where they struggle, and how teachers can help them better. We used submission history from the Ingenious¹ platform, and a real survey to see first-year students learning how to program from different angles.

7.1 Answer to RQ1: What behavioral profiles exist?

We found two main student groups in the submission data. Those groups contain profiles built from four simple metrics: how much students try, how well they do, how fast they retry, and how often they take long breaks. These four measures create **16 possible profiles**.²

In practice, the two big groups separate students into a stronger group and a weaker group. The stronger group tends to do well, solve more questions, and keep moving with fewer long breaks. The weaker group struggles more, gets lower scores, and often needs more tries. We showed that the big split is useful, but the 16 smaller profiles give a more detailed view, as shown in Table D.2 and Fig 4.6. One important fact is that student behavior changes from one question to another.³ A student can do very well on one question and still struggle on another.

¹Ingenious autograder platform: https://docs.ingenious.org/en/latest/what_is_ingenious.html

²These profiles come from combining the four yes/no dimensions defined in Chapter 4: persistence, effectiveness, iteration speed, and engagement.

³This is one of the most useful findings of the thesis: profiles help, but they do not freeze a student into one label.

7.2 Answer to RQ2: Why do students adopt these behaviors?

The survey and the code examples show some interesting reasons. The hardest topic in the survey was *loops and algorithm*, as shown in Table 6.2 and Figure 6.2. Question about *File reading* was also harder for respondents, especially in the Bachelor 1 results shown in Figure 6.3. In the code cases analysis from Chapter 5, we saw three common behaviors: some students had a clear plan and moved forward step by step, some kept trying but repeated the same mistake, and some rushed or stopped early when they felt lost.

In short, sometimes students know the idea but not the method. Sometimes they just need help breaking the problem into smaller steps and sometimes they need a better way of assessing their own code.

7.3 Answer to RQ3: How do profiles and strategies align?

The quantitative and qualitative analysis mostly goes in the same direction. Students who do well usually have cleaner code and a more planned approach. Students who struggle often jump into code too fast, repeat the same wrong idea, or stop too early. However, we have few exceptions. Sometimes, even student from satisfactory profiles have a non optimal code quality (they focus on correctness, not on structural quality of the code), and in the other sides, some students have a seemingly good code structure even though it might not perform the best.

7.4 Limits of this study

This work studies one course, one exam session, and one survey. So we should be careful not to claim that every programming class will look the same. The survey also depends on what students said about their work, which may not always match what they actually did.⁴

We also have less data for some student levels than for others, so the strongest conclusions are for Bachelor 1 and Master students.

⁴That is why the survey is useful, but not enough on its own.

7.5 Future work

There are potential good next steps: First, we could study whether the teaching tips we propose in this thesis, actually help students in a real classes. Second, we could test more cluster sizes, especially higher values of k , to see whether we can get a cleaner separation picture (alongside its pedagogical benefit). Third, we could repeat the study on more years to see if the same patterns stay stable.

It would also be useful to compare other programming courses that approach other languages like Java, because students may struggle in different ways depending on the setting.

References

- [1] Ryan S. Baker and Paulo S. Inventado. “Educational Data Mining and Learning Analytics”. In: *Learning Analytics* (2014), pp. 61–75.
- [2] John W. Creswell and Vicki L. Plano Clark. *Designing and Conducting Mixed Methods Research*. 3rd ed. Sage, 2017. ISBN: 978-1-5063-3898-0.
- [3] Petri Ihantola, Arto Vihavainen, Alireza Ahadi, et al. “Educational Data Mining and Learning Analytics in Programming: Literature Review and Case Studies”. In: *Proceedings of ITiCSE Working Group Reports* (2015), pp. 41–63.
- [4] Tony Jenkins. “On the Difficulty of Learning to Program”. In: *Proceedings of the 3rd Annual Conference of the LTSN Centre for Information and Computer Sciences*. Available via conference proceedings; indexed in Google Scholar. 2002, pp. 53–58.
- [5] Leonard Kaufman and Peter J. Rousseeuw. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley, 1990.
- [6] Cazembe Kennedy and Eileen T. Kraemer. “Qualitative Observations of Student Reasoning: Coding in the Wild”. In: *Proceedings of the 2019 ACM Conference on Innovation and Technology in Computer Science Education*. ITiCSE ’19. New York, NY, USA: Association for Computing Machinery, 2019, pp. 224–230. DOI: [10.1145/3304221.3319751](https://doi.org/10.1145/3304221.3319751).
- [7] Essi Lahtinen, Kirsti Ala-Mutka, and Hannu-Matti Järvinen. “A Study of the Difficulties of Novice Programmers”. In: *Proceedings of ITiCSE*. 2005, pp. 14–18.
- [8] Chris Piech et al. “Deep Knowledge Tracing”. In: *Advances in Neural Information Processing Systems*. 2015, pp. 505–513.
- [9] Anthony Robins, Janet Rountree, and Nathan Rountree. “Learning and Teaching Programming: A Review and Discussion”. In: *Computer Science Education* 13.2 (2003), pp. 137–172.

- [10] Cristóbal Romero and Sebastián Ventura. “Educational Data Mining: A Review of the State of the Art”. In: *IEEE Transactions on Systems, Man, and Cybernetics, Part C* 40.6 (2010), pp. 601–618.
- [11] Peter J. Rousseeuw. “Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis”. In: *Journal of Computational and Applied Mathematics* 20 (1987), pp. 53–65.
- [12] Joe H. Ward. “Hierarchical Grouping to Optimize an Objective Function”. In: *Journal of the American Statistical Association* 58.301 (1963), pp. 236–244.
- [13] Christopher Watson and Frederick W. B. Li. “Failure Rates in Introductory Programming Revisited”. In: *Proceedings of ITiCSE Working Group Reports* (2014), pp. 39–68.
- [14] Barry J. Zimmerman. “Becoming a Self-Regulated Learner: An Overview”. In: *Theory Into Practice* 41.2 (2002), pp. 64–70. DOI: [10.1207/s15430421tip4102_2](https://doi.org/10.1207/s15430421tip4102_2).

Appendix A

Reproducibility Appendix

Pipeline map

The analysis pipeline is: `extraction` $\rightarrow$ `preprocessing` $\rightarrow$ `clustering` $\rightarrow$ `profile assignment` $\rightarrow$ `analytics`.

Thresholds used for profiling

The current profile thresholds are:

- `avg_attempts` ≥ 15
- `avg_best_score` ≥ 50
- `avg_fast_retry_ratio` ≥ 0.50
- `avg_long_pause_ratio` ≤ 0.10

Cluster choice

The current partition is $k = 2$. Cluster selection is checked with silhouette scores and cluster sizes, and the full numerical details are kept in Chapter 4.

Appendix B

Additional Visualizations

Archived figures

The figure below is not needed to follow the main argument, but can be useful if a reader wants to compare alternatives.



Figure B.1: Alternative clustermap.

Appendix C

Statistical Tables

Top question-pair correlations

Table C.1: Top absolute correlations of success and best-score between questions.

Pair	Success	Best score
$q1, q2$	0.495	0.526
$q3, q4$	0.491	0.524
$q2, q3$	0.478	0.498
$q1, q3$	0.473	0.492
$q1, q4$	0.410	0.462

Cluster choice summary

The full elbow calculation is kept short here because Chapter 4 already explains the method. The compact summary is:

- the elbow appears around $k = 3$,
- $k = 2$ is still kept because it is easier to read and only slightly worse on silhouette,
- Full numeric table is left in the main chapter.

Appendix D

Glossary

This glossary keeps only the terms that help the reader follow the thesis quickly.

Abbreviations

RQ1: What behavioral profiles exist?

RQ2: Why do students adopt these behaviors?

RQ3: How do profiles and strategies align?

INGInious: The online judge used to collect submission traces.

EDM: Educational Data Mining.

GDPR: General Data Protection Regulation.

Principal features

- `avg_best_score / median_best_score`: highest score reached by a student, averaged or summarized by median.
- `avg_attempts / median_attempts`: how many tries students make before moving on.
- `avg_fast_retry_ratio`: share of quick resubmissions.
- `avg_long_pause_ratio`: share of resubmissions after longer breaks.
- `solved_rate`: proportion of questions solved.

Engineered feature table

This table is the compact reference used throughout the thesis for the aggregated features built from submission traces.

Table D.1: Overview of basic measurements and student behaviors we look at

Dimension	Feature / Signal Name	What it captures
Performance	<i>solved_ratio</i>	Share of exercises successfully solved.
	<i>[avg / median]_best_score</i>	Typical highest score reached per exercise.
	<i>[avg / median]_final_score</i>	Typical final score recorded per exercise.
	<i>[avg / median]_score_gain</i>	Change from start to finish.
Effort &	<i>[avg / median]_attempts</i>	Typical number of tries per exercise.
Persistence	<i>total_attempts</i>	Total number of submissions.
	<i>[avg / median]_improving_ratio</i>	Share of attempts that improved the score.
	<i>never_solved_count</i>	Number of exercises left unsolved.
Temporal	<i>[avg / median]_time_to_success_sec</i>	Time needed to reach success.
Dynamics	<i>[avg / median]_span_sec</i>	Total time between first and last attempt.
	<i>[avg / median]_mean_delta_sec</i>	Mean time between submissions.
	<i>[avg / median]_fast_retry_ratio</i>	Share of retries within short gaps.
	<i>[avg / median]_long_pause_ratio</i>	Share of retries after long gaps.
Synthetic	mastery_signal	Combined signal for success and stability.
Signals	attempts_volume_signal	Combined signal for volume of effort.
	timing_median_signal / _avg_signal	Combined tempo signal.

Clustering terms

Hierarchical clustering: a method that groups similar students step by step.

Dendrogram: the tree diagram produced by hierarchical clustering.

Silhouette score: a check on how well the groups separate.

Cut level: the point where the tree is split into clusters.

Behavioral dimensions

- **Persistence:** do students keep trying?
- **Effectiveness:** do they reach good scores?
- **Iteration speed:** do they retry quickly?
- **Sustained engagement:** do they come back after breaks?

The 16 profile table

The thesis uses four binary dimensions to build the 16 profile labels below. The thresholds are the same ones used in Chapter 4 and referenced throughout the report.

Table D.2: Complete 16 profiles and thresholds used in the thesis

Profile Name	P	E	I	S	Interpretation
Elite Learner	H	H	H	H	Strong effort, high success, iterative, engaged
Effective Sprint	H	H	H	L	Strong effort and success, iterative, episodic
Methodical	H	H	L	H	Strong effort and success, methodical, engaged
Lucky Guesser	H	H	L	L	High success with low iteration, possible guessing
Persistent Learner	H	L	H	H	Strong effort but lower success, iterative, engaged
Thrashing	H	L	H	L	Many attempts, low success, rapid retries, episodic
Stuck Thinker	H	L	L	H	Strong effort but low success, minimal retry, engaged
Struggling Guesser	H	L	L	L	Strong effort, low success, minimal retry, episodic
Efficient Solver	L	H	H	H	Minimal effort, high success, iterative, engaged
Quick Success	L	H	H	L	Quick high scores, iterative, episodic
Confident	L	H	L	H	High success, minimal iteration, methodical, engaged
Lucky (High)	L	H	L	L	High success with minimal effort and iteration
Resigned	L	L	H	H	Low effort, low success, but iterative and engaged
Quick Quit	L	L	H	L	Low effort and success, some iteration, episodic
Passive	L	L	L	H	Low effort and success, minimal iteration, engaged
Checked Out	L	L	L	L	Minimal effort, low success, no iteration, episodic

Threshold note: P = Persistence (≥ 15 attempts), E = Effectiveness ($\geq 50\%$ score), I = Iteration (≥ 0.50 fast-retry ratio), S = Sustained engagement (≤ 0.10 long-pause ratio).

Profile words used in the thesis

The thesis uses short profile names such as *Elite Learner*, *Persistent Learner*, *Quick Success*, and *Checked Out*.

UNIVERSITÉ CATHOLIQUE DE LOUVAIN
École polytechnique de Louvain

Rue Archimède, 1 bte L6.11.01, 1348 Louvain-la-Neuve, Belgique | www.uclouvain.be/epl